

University of Algiers 1
Faculty of Science
Department of Computer Science

Strata

Design and Deployment of a Multi-Site IaaS Provider Infrastructure

Project Report — Network & Systems Administration

Abstract

Cloud providers face a problem most enterprises don't: they have to run their own internal business and host someone else's infrastructure on the same physical hardware at the same time, and a mistake in either direction is expensive. This report documents Strata, a fictional multi-site Infrastructure-as-a-Service provider built to solve that problem properly — with a hard boundary between staff and customers, no single point of failure at any layer, and self-service compute a customer can actually provision without asking anyone. It covers the two-site network design, the redundancy built into every tier, perimeter and inter-site security — the zone-to-zone ACL policy and IDS sensor deployment — the core infrastructure services, the automation that runs the whole thing, and the two applications built on top of it. The report stops at design, deployment, and detection; the attack-simulation campaign against this infrastructure is run and documented separately, as its own exercise.

1. Introduction and Project Context

Cloud providers have a structural problem that ordinary enterprises don't face: they run their own internal business and host someone else's infrastructure on the same hardware, at the same time — and a mistake in either direction is expensive. Let a customer's traffic reach the internal network, and every customer becomes a potential path into payroll and HR data. Let one router or one firewall carry an entire site, and a single failed card takes down every customer at once. Let provisioning stay a manual, ticket-and-wait process, and the "cloud" is really just a slower version of a normal hosting company.

Strata is a fictional IaaS provider built to solve exactly those three problems: strict separation between staff and customers, no single point of failure anywhere in the path, and self-service compute that customers can provision themselves. Like any real cloud operator, it has to run its own corporate operations while simultaneously hosting and exposing compute resources sold to external customers — an internal enterprise network alongside a customer-facing public cloud, with the two populations kept strictly apart at every layer, while both are still served by one coherent, centrally managed and automated infrastructure.

1.1 Scope

Strata's infrastructure is split across two physically distinct sites, connected over an untrusted ISP transit:

- Headquarters (HQ) hosts Strata's internal corporate network and the operational systems used by its ~20 staff, organized into an IT, HR, and Business-Operations department.
- Cloud Datacenter hosts the customer-facing IaaS infrastructure: the compute fabric, the customer portal, and the workloads consumed by external customers.

This report covers the network, systems, automation, and application work on that infrastructure — everything needed to take both sites from a blank topology to a working, self-service IaaS platform with a fully separated internal network, an explicit ACL policy, and IDS coverage at the two mandatory sensor positions. The attack-simulation campaign against this infrastructure, and its results, is documented separately as its own exercise.

2. System Architecture

2.1 What goes wrong without this design

Before laying out the topology, it's worth walking through what actually breaks when a multi-tenant cloud isn't built carefully — because every decision below exists to prevent one of these failures, not to satisfy a checklist.

- No strict separation between populations: a compromise anywhere in the public-facing side — the portal, a customer VM, the DMZ web server — becomes a path straight into the corporate network. One breached customer workload should never be able to touch payroll, HR records, or staff mail.
- No redundancy above the access layer: a single firewall, router, or switch failure takes an entire zone offline. In a real cloud, that means every customer behind it loses service at once, not just one connection.
- A trusted transit link between sites: if the ISP circuit between HQ and the Datacenter is assumed to be safe, anyone with access to that transit path can read or tamper with everything crossing it — including provisioning traffic and internal mail.
- Manually provisioned compute: if spinning up a customer VM requires a human to log in and click through steps, the platform isn't really a cloud — it's a hosting company with extra branding, and it can't scale past what one operator can click through in a day.

The project brief distilled these four failure modes into four requirements that shaped every subsequent design decision:

- Strict population separation — a compromise of any public-facing datacenter component must never translate into access to the corporate network, and no customer traffic may ever reach HQ.
- No single point of failure — covering hardware links, network paths, gateways, and data, not just one layer in isolation.
- Multi-site interconnection over an untrusted transit — the ISP link between sites can't be trusted, so confidentiality has to be enforced by the endpoints, not the transit network.
- Self-service compute — the datacenter side has to behave like an actual cloud: programmatically provisionable, not a static set of pre-built servers.

2.2 Site layout

Both sites connect to the public internet through their own independent ISP links, and to each other through a site-to-site interconnection terminated at each site's perimeter gateway (Chapter 5).

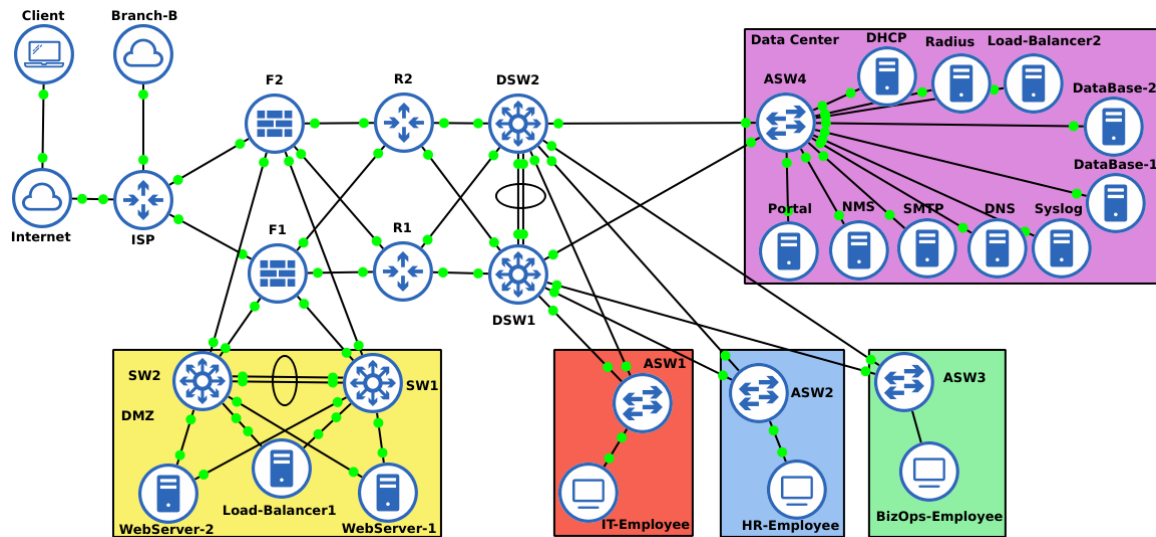


Figure 2.1: Headquarters (Branch A) — full conceptual design, redundant three-tier campus.

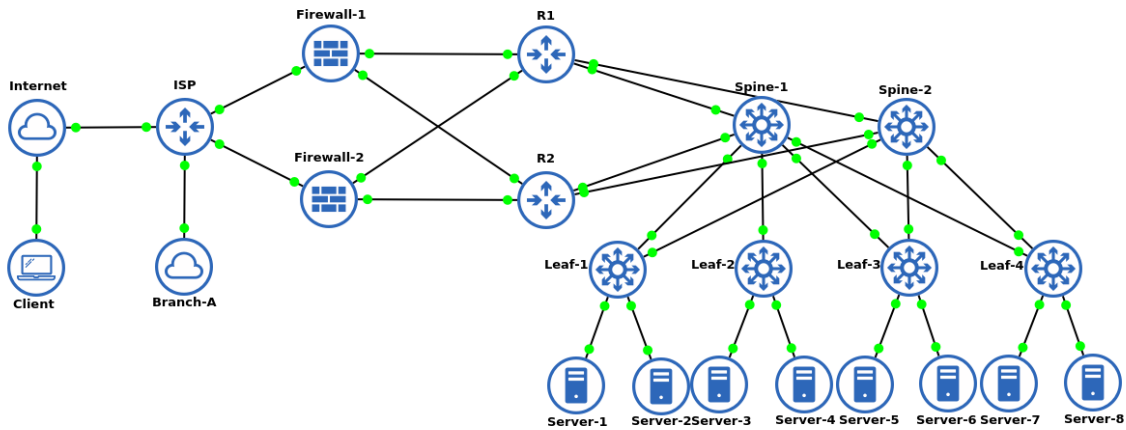


Figure 2.2: Cloud Datacenter (Branch B) — full conceptual design, spine-leaf compute fabric.

Headquarters follows a classic redundant three-tier enterprise design: a dual-firewall perimeter (F1/F2), a dual-router core (R1/R2), a dual distribution-switch layer (DSW1/DSW2), and an access layer split by department (IT/HR/BizOps), plus a DMZ hosting the employee-facing web tier and a Data Center segment hosting shared internal services (DHCP, RADIUS, mail, DNS, monitoring, load balancing, databases). Every tier above the access layer is built as a redundant pair with full-mesh links between adjacent tiers, so no single link or device failure isolates a zone.

Cloud Datacenter is built around a spine-leaf Clos fabric behind its own redundant firewall/router pair: two spine switches form the fabric core with no direct spine-to-spine link, and each leaf switch connects to every spine, giving any two leaves at least two independent paths between them. Customer workload hosts — VMs and containers — attach to the leaves.

2.2.1 As-built topology

Figures 2.1 and 2.2 show the full conceptual design — the redundant pairs at every tier that the "no single point of failure" requirement calls for. The lab hardware available for this project couldn't run that many router/switch VMs simultaneously alongside the customer workloads the platform needed to host, so both sites were actually built and validated with a single device per tier instead of a redundant pair — a real

hardware constraint, not a design walk-back. In production, on real hardware, the conceptual design in 2.1/2.2 is what would actually run.

The redundancy mechanisms themselves — HSRP, LACP, and OSPF's convergence behaviour — were designed and validated separately in Packet Tracer against the full dual-device topology before the project moved to GNS3/Containerlab for the parts that mattered most for this report: real routing between sites, and a real, working provisioning pipeline. Chapter 3's HSRP and LACP configurations (3.3, 3.4) are exactly what was validated in Packet Tracer; the GNS3 build referenced from Chapter 3 onward runs the single-device version shown below.

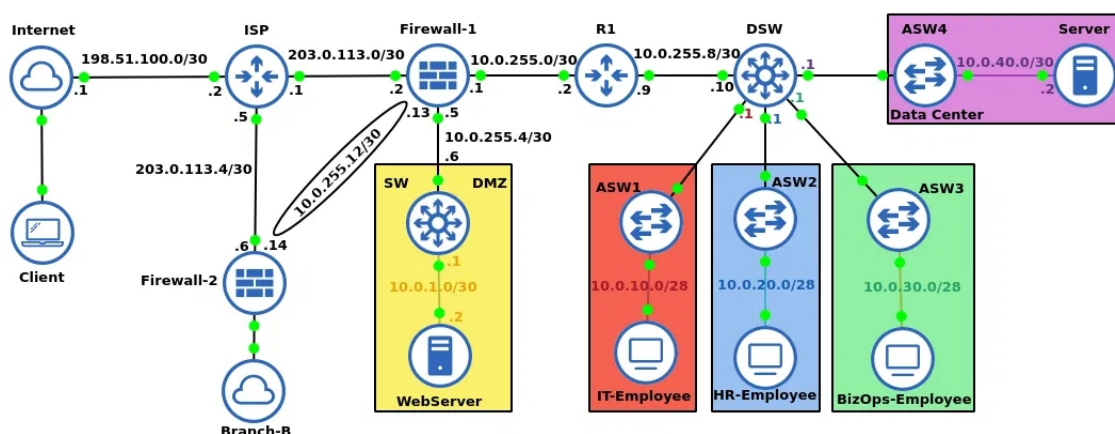


Figure 2.3: Headquarters — as-built GNS3 topology (single firewall, router, and distribution switch per tier; Firewall-2 is the Datacenter's own perimeter gateway, reached over the inter-site link).

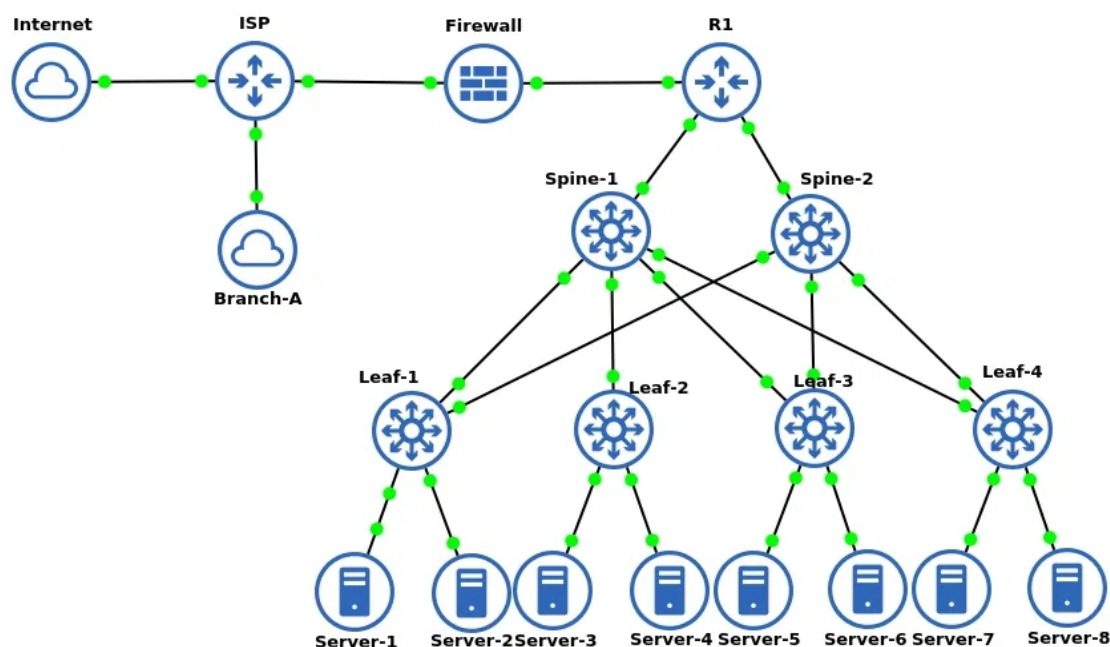


Figure 2.4: Cloud Datacenter — as-built GNS3/Containerlab topology. Unlike the HQ side, the spine-leaf fabric itself was built at full target scale: two spines and four leaves, each leaf uplinked to both spines, hosting eight customer-facing servers.

2.3 Network segmentation

Both sites enforce a mandatory zone model, with every inter-zone flow passing through a stateful firewall — this is the direct answer to the "strict population separation" failure mode above:

- Public / DMZ zone (Datacenter) — the externally reachable layer hosting the customer portal and its API. Treated as hostile territory: a compromise here must not grant access to customer workloads, the database tier, or HQ.
- Customer workload zone (Datacenter) — the spine-leaf fabric hosting customer compute. Reachable from the DMZ only for the strict minimum traffic the provisioning control plane needs.
- Internal corporate network (HQ) — accessible exclusively to Strata staff, hosting the employee portal and directory service. No path exists from the Datacenter DMZ into this zone.
- Backend data zone — the database tier, unreachable from the DMZ; only the appropriate application servers may connect to it.

An explicit, documented zone-to-zone policy governs every permitted flow (Section 5.1.3); everything else is denied by default at the gateway.

2.4 Why this architecture

2.4.1 Redundant pairs, not single devices, above the access layer

A single firewall, router, or distribution switch at HQ would put the whole site one hardware failure away from an outage — pull one cable, or have one device crash, and everything behind it goes dark. That's why F1/F2, R1/R2, and DSW1/DSW2 are each built as active pairs, full-meshed to the tier above and below: losing any one device, or any one link between two tiers, never isolates a zone.

2.4.2 HSRP, load-shared rather than pinned to one switch

Running both distribution switches as a plain active/standby pair would leave one of them sitting idle under normal conditions — wasted capacity for no benefit until something actually breaks. Splitting HSRP priority so DSW1 is active for VLANs 90/100/200 and DSW2 is active for VLANs 300/400 keeps both switches forwarding real traffic simultaneously, while either one can still absorb the other's full load the moment its partner fails.

2.4.3 Spine-leaf instead of a traditional core/distribution/access stack

Customer workloads in the Datacenter are created, moved, and destroyed constantly — nothing like HQ's fixed department VLANs, which barely change from month to month. A traditional hierarchical design assumes a stable layout; a spine-leaf Clos fabric doesn't need one. Every leaf reaches every other leaf through either spine at equal cost, so growing the fabric just means adding leaves and spines rather than redesigning a tree, and losing a single spine never splits the fabric in two.

2.4.4 A dedicated stateful gateway instead of router ACLs

Router-level ACLs live scattered across whatever interface each rule happens to sit on, which makes the overall security picture hard to audit — you'd have to check every device to know what's actually allowed. A dedicated firewall (pfSense) puts the entire zone policy in one place, tracks connection state so return traffic doesn't need its own mirrored rule, and is also where NAT and the VPN terminate — one component to reason about for the whole security boundary, instead of many.

2.4.5 OSPF instead of static routing

With redundant devices and redundant links everywhere, static routing would mean manually updating a routing table on every device, every time the topology changed even slightly. OSPF converges around a failed link or device on its own — which is the only way the "no single point of failure" goal actually holds up in practice, rather than just on paper.

3. Network Design — Headquarters

As noted in 2.2.1, the configurations below — OSPF, HSRP, and LACP — were designed and validated against the full dual-device topology in Packet Tracer; the GNS3 build these sections otherwise describe runs a single router and single distribution switch per the hardware-limited as-built topology (Figure 2.3). They're kept in full here because they're what the redundant design in Figures 2.1/2.2 would run on real hardware.

3.1 Addressing plan

The whole internal address space is drawn from 10.0.0.0/16. Every VLAN subnet and HSRP gateway, every point-to-point link, and every device's loopback is documented interface-by-interface in Addressing/ (IPv4 Addresses.pdf, Connections.pdf), which is the authoritative source of truth for addressing rather than duplicated here.

Every point-to-point inter-device link (firewall-router, router-switch, etc.) gets its own /30, and every device carries a /32 loopback used as its OSPF router-id and management identity. One public IPv4 (203.0.113.8/32) is reserved for the DMZ load balancer.

3.2 Routing

OSPF area 0 runs throughout HQ. Every inter-device link is tuned as point-to-point so there's no unnecessary DR/BDR election overhead on links that only ever have two neighbors — that election exists to handle broadcast segments with many routers, which a direct link between two devices never has:

R1 — OSPF process and loopback advertisement

```
router ospf 1
  passive-interface Loopback0
  network 10.0.0.3 0.0.0.0 area 0
  network 10.0.255.32 0.0.0.3 area 0
  network 10.0.255.36 0.0.0.3 area 0
!
interface GigabitEthernet0/1
  ip address 10.0.255.33 255.255.255.252
  ip ospf network point-to-point
!
ip route 0.0.0.0 0.0.0.0 10.0.255.1
```

The firewalls originate a default route toward the ISP; every other prefix is learned dynamically through OSPF, so adding a new device or a new redundant path only needs an OSPF adjacency — never a manual static-route update somewhere else in the fabric.

3.3 First-hop redundancy: HSRP

Every VLAN SVI on the distribution-switch pair runs HSRP, load-shared as described in 2.4.2 rather than pinned to a single active switch:

DSW1 — VLAN 100 SVI with HSRP (active for this VLAN) and DHCP relay

```
interface Vlan 100
ip address 10.0.10.2 255.255.255.240
ip helper-address 10.0.40.2
standby 10 ip 10.0.10.1
standby 10 priority 110
standby 10 preempt
```

DSW2 — same VLAN, standby role here; priorities reverse on VLANs 300/400 so DSW2 is active there instead

```
interface Vlan 100
ip address 10.0.10.3 255.255.255.240
ip helper-address 10.0.40.2
standby 10 ip 10.0.10.1
standby 10 priority 90
standby 10 preempt
```

Every user VLAN SVI carries ip helper-address 10.0.40.2, relaying DHCP requests from access-layer clients to the Data Center DHCP server regardless of which distribution switch currently holds the HSRP-active role for that VLAN.

3.4 Link redundancy: LACP EtherChannel

The distribution-switch pair and the DMZ switch pair are each bundled with an LACP EtherChannel across two physical uplinks, so a single link failure between either pair has zero impact on connectivity or on the HSRP adjacency between them:

DSW1 — Port-channel1, LACP active mode, bundling two Gi uplinks to DSW2

```
interface Port-channel1
switchport mode trunk
!
interface range GigabitEthernet1/2 - 3
channel-group 1 mode active
switchport mode trunk
```

3.5 Layer-2 hygiene at the access layer

Rapid-PVST runs globally, with portfast and bpduguard enabled on every access port, so a rogue switch plugged into an access port can't inject BPDUs into the spanning-tree topology. This is pushed by Ansible (Section 7.1), not configured by hand per device.

4. Cloud Datacenter Compute Fabric

4.1 Spine-leaf topology

The Datacenter's customer-workload network follows the Clos topology mandated by the project brief: two spines, no direct spine-to-spine link, and every leaf connected to every spine, so ECMP spreads traffic across both spine uplinks and the loss of a single spine never partitions any pair of leaves.

4.2 Implementation as a Containerlab topology

Instead of running the Datacenter fabric as full router/switch VMs — the way HQ's build does — it's implemented with Containerlab (`Config/Devices/Containerlab/Branch-B.yml`). That keeps it light enough to run alongside the customer VMs and containers it hosts, on the same hypervisor, while still exercising the same OSPF/ECMP behaviour a physical build would show. Unlike HQ, this side of the lab was built at full target scale rather than simplified for hardware headroom: two spines and four leaves, each leaf uplinked to both spines, hosting eight customer-facing servers (Figure 2.4) — the same topology the spine-leaf rationale in Chapter 2 assumes.

Branch-B — Containerlab topology (topology.yml): r2 uplinks to both spines, each leaf uplinks to both spines, and each leaf's third interface bridges into the hypervisor's VM fabric.

```
name: Branch-B
topology:
kinds:
  ceos:
    image: ceos:4.34.1F
  cisco_iol:
    image: vrnetlab/cisco_iol:17.18.02
  linux:
    image: nat-node:alpine
nodes:
  r2:
    kind: cisco_iol
    startup-config: r2.txt.partial
  nat:
    kind: linux
    network-mode: none
  spine1:
    kind: ceos
    startup-config: spine1.txt.partial
  spine2:
    kind: ceos
    startup-config: spine2.txt.partial
  leaf1:
    kind: ceos
    startup-config: leaf1.txt.partial
  leaf2:
    kind: ceos
    startup-config: leaf2.txt.partial
  leaf3:
    kind: ceos
    startup-config: leaf3.txt.partial
  leaf4:
    kind: ceos
    startup-config: leaf4.txt.partial
  gns3-br:
    kind: bridge
  br-fabric:
    kind: bridge
links:
  - endpoints: ["r2:e0/1", "gns3-br:eth0"]
  - endpoints: ["r2:e0/2", "spine1:eth1"]
  - endpoints: ["r2:e0/3", "spine2:eth1"]
  - endpoints: ["spine1:eth2", "leaf1:eth1"]
  - endpoints: ["spine1:eth3", "leaf2:eth1"]
  - endpoints: ["spine1:eth4", "leaf3:eth1"]
  - endpoints: ["spine1:eth5", "leaf4:eth1"]
  - endpoints: ["spine2:eth2", "leaf1:eth2"]
  - endpoints: ["spine2:eth3", "leaf2:eth2"]
  - endpoints: ["spine2:eth4", "leaf3:eth2"]
  - endpoints: ["spine2:eth5", "leaf4:eth2"]
  - endpoints: ["leaf1:eth3", "br-fabric:eth0"]
```

- endpoints: ["leaf2:eth3", "br-fabric:eth1"]
- endpoints: ["leaf3:eth3", "br-fabric:eth2"]
- endpoints: ["leaf4:eth3", "br-fabric:eth3"]

leaf — server-facing interface: OSPF area 1, DHCP relay to HQ

```
interface Ethernet2
ip address 10.10.10.1/24
ip helper-address 10.0.40.2
no switchport
!
router ospf 1
router-id 10.0.0.8
area 0.0.0.1
```

spine — uplink to r2 and downlink to leaf, both OSPF point-to-point

```
interface Ethernet0/1
ip address 10.0.255.18/30
ip ospf network point-to-point
!
interface Ethernet0/2
ip address 10.0.255.25/30
ip helper-address 10.0.40.2
!
router ospf 1
router-id 10.0.0.6
```

Every customer VM or container's default gateway resolves to the leaf's Ethernet2 address, and every DHCP lease is relayed all the way back to the same central DHCP server used at HQ — the Datacenter fabric is a genuinely separate site, but it shares the same IPAM and DNS infrastructure rather than duplicating it.

5. Perimeter Security and Site Interconnection

5.1 Gateway and firewall design

Each site has its own redundant pfSense pair at its perimeter — F1/F2 at HQ, Firewall-1/Firewall-2 at the Datacenter — acting as the dedicated gateway for every inter-zone flow at that site: routing between the external network, the local zones, and the inter-site link; stateful, default-deny rules between every zone pair; NAT for outbound traffic and inbound access to published services; and logging of denied flows for later reconstruction. pfBlockerNG was evaluated for DNS/IP-based blocking early on and later removed from the design; it is not part of the current build.

5.1.1 HQ perimeter firewall (F1/F2)

Inbound rule set, source = outside (WAN):

- Permit HTTP to the DMZ (the customer-facing web tier reachable at HQ).
- Permit all traffic over the site-to-site VPN tunnel (already authenticated and encrypted at the tunnel itself).
- Deny any — no other unsolicited inbound flow is accepted; normal outbound/NAT'd internet access for internal hosts is unaffected by this inbound rule set.

5.1.2 Datacenter perimeter firewall (Firewall-1/Firewall-2)

Inbound rule set, source = outside (WAN):

- Permit SSH to the customer workload zone (the VPS/spine-leaf fabric) — this is how customers reach the VMs they provision, via the NAT gateway's public-pool address for each VM.
- Permit all traffic over the site-to-site VPN tunnel.
- Deny any — same default-deny posture as the HQ firewall, scoped to this site's own WAN edge.

Everything in Section 5.1.3 below is inter-zone policy enforced once traffic is already inside one of the two sites — behind these two WAN-facing rule sets, not instead of them.

5.1.3 Zone-to-zone policy

The policy actually implemented — translated into pfSense firewall rules at the relevant interfaces — is a default-deny policy with a short, explicit allow-list. Every permitted flow below corresponds to a concrete rule scoped to a specific interface, protocol, and destination, rather than a broad zone-to-zone allow:

Source	Permitted destination / service	Enforced at
IT department (VLAN 100)	Data Center zone, full access	Internal ACL — DSW1/DSW2 inter-VLAN routing
All departments (VLAN 100/200/300)	Internal portal, HTTP only	Internal ACL — DSW1/DSW2 inter-VLAN routing
Any internal client, and Datacenter fabric via the site-to-site link	DHCP/67 and DNS/53 to the DHCP/DNS server in HQ's Data Center zone	Firewall — HQ F1/F2 for intra-HQ crossings; Datacenter Firewall-1/Firewall-2 for the inter-site leg

Source	Permitted destination / service	Enforced at
DMZ web server	Internal database, PostgreSQL/5432 only — nothing else	Firewall — F1/F2 (DMZ and Data Center zone only connect through the firewall pair, not through DSW1/DSW2)
External / DMZ clients	No path into the internal corporate network	Firewall — default-deny on both perimeter firewalls
IT department	Branch office link (site-to-site tunnel)	Firewall — VPN tunnel terminates on F1/F2 and Firewall-1/Firewall-2
DMZ / Data Center	Branch office link, limited to centralized monitoring/logging (Syslog, SNMP) — nothing else	Firewall — VPN tunnel

5.2 Site-to-site interconnection

Branch sites are interconnected over an Ethernet WAN service provided by the ISP. Because that transit infrastructure sits outside organizational control, a site-to-site IPsec VPN runs between the two sites' perimeter firewalls to guarantee confidentiality, integrity, and authentication of everything crossing it. The tunnel terminates directly at each site's gateway, so a tunnel failure simply removes the inter-site path rather than exposing anything in plaintext, and firewall rules are enforced at both tunnel endpoints so only traffic explicitly permitted by the zone policy above ever crosses it.

5.3 Intrusion detection (Suricata on pfSense)

Suricata runs directly on pfSense's WAN interface rather than as a separate appliance, in IDS mode with blocking disabled — it inspects and alerts on traffic, it does not drop it, so a misfiring signature can never take down legitimate service.

5.3.1 Packet-capture accuracy

Hardware checksum offloading, TCP segmentation offloading (TSO), and large receive offloading (LRO) were disabled on the interface (System → Advanced → Networking) before enabling the sensor. Left on, the NIC driver can hand Suricata packets that have already been coalesced or checksum-modified in hardware, which breaks signature matching against the actual on-wire packet; disabling offload guarantees Suricata inspects packets exactly as they arrived.

5.3.2 Interface, ruleset, and logging

The WAN-interface instance runs promiscuous, with HTTP logging and performance statistics enabled and the default Home Net / External Net definitions kept, and its own event log is forwarded into pfSense's system log — the same syslog path already centralized into Graylog (Section 6.4), so IDS alerts land in the same searchable timeline as every other device's logs rather than a separate silo. Detection rules come from the Emerging Threats Open (ET Open) ruleset, kept current through Suricata's built-in automatic rule updates.

5.3.3 Enabled rule categories

Only the categories relevant to the project's planned attack scenarios are active — emerging-scan, emerging-sql, emerging-web_server, emerging-web_client, emerging-exploit, and related categories — rather than the full ET Open set, keeping alert volume focused on what's actually being tested against. Rule counts were verified per category after the update (emerging-scan alone loaded 793 signatures), confirming the ruleset was actually in place before relying on it.

5.3.4 Validation

An Nmap port scan against the firewall's full port range, and a direct SSH connection attempt to it, were used as a first functional test of the sensor — not one of the project's formal attack scenarios, just confirmation the pipeline works end to end. Both were correctly detected, with matching alerts appearing in Suricata's Log View (Alerts) interface, confirming the sensor is live and correctly wired before it is relied on for the actual test campaign.

6. Core Infrastructure Services

All core services run in the HQ Data Center zone and are reachable by the rest of the internal network only on the ports they need.

Function	Software
DHCP	ISC dhcpd
DNS	ISC BIND9
AAA / RADIUS	FreeRADIUS + PostgreSQL
SMTP	Postfix
IMAP	Dovecot
SNMP / metrics	LibreNMS
Centralized logging	Graylog (MongoDB-backed)

Function	Software
Config management	Ansible
Compute provisioning	Terraform (libvirt, Docker)

6.1 DHCP and dynamic DNS

dhcpd serves one pool per VLAN (for example 10.0.10.4–10.0.10.14 for IT, 600 s default / 7200 s max lease), handing out the HSRP VIP of the relevant VLAN as the default gateway. Every lease triggers a secure Dynamic DNS update to BIND, authenticated with a TSIG key (HMAC-SHA256) so only the DHCP server can write records into the zone:

/etc/dhcp/dhcpd.conf — TSIG key and DDNS zone declarations (secret redacted)

```
key "dhcp-key" {
    algorithm hmac-sha256;
    secret "<redacted>";
};

ddns-update-style interim;
ddns-domainname "pfe2627.xyz.";
ddns-rev-domainname "in-addr.arpa.";

zone pfe2627.xyz. {
    primary 10.0.40.5;
    key dhcp-key;
}
zone 10.0.10.in-addr.arpa. {
    primary 10.0.40.5;
    key dhcp-key;
}
```

/etc/bind/named.conf.local — matching allow-update on the BIND side

```
include "/etc/bind/dhcp.key";

zone "pfe2627.xyz" {
    type master;
    file "/var/cache/bind/db.pfe2627.xyz";
    allow-update { key dhcp-key; };
};
zone "10.0.10.in-addr.arpa" {
    type master;
    file "/var/cache/bind/db.10.0.10";
    allow-update { key dhcp-key; };
};
```

Four matching reverse zones (one per VLAN subnet) keep PTR records in sync the same way, so every dynamically leased address is both forward- and reverse-resolvable with no manual DNS maintenance. In the Cloud Datacenter, DHCP additionally drives the NAT-automation pipeline described in 8.1.1 — an on-commit hook fires for every lease to allocate and install a public-IP mapping for the new workload.

6.2 AAA

FreeRADIUS is backed by a PostgreSQL database rather than flat files, so authentication and accounting records are queryable and centrally managed. Connection limits are capped (`max_connections = 16`, `idle_timeout = 30`) to bound resource usage from any single NAS client.

6.3 Mail

Postfix and Dovecot together provide internal-plus-external mail:

/etc/postfix/main.cf — internal relay networks, external TLS relay, Dovecot-delegated SASL

```
myhostname = mail.pfe2627.xyz
mydomain = pfe2627.xyz
mynetworks = 127.0.0.0/8, 10.0.10.0/24, 10.0.20.0/24, 10.0.30.0/24
relayhost = [smtp-relay.brevo.com]:587
smtp_sasl_auth_enable = yes
smtp_tls_security_level = encrypt
smtpd_sasl_type = dovecot
smtpd_sasl_path = private/auth
```

Postfix handles virtual mailbox routing for the `pfe2627.xyz` domain (a dedicated `vmail` system account, `uid/gid 5000`) and relays outbound mail through an authenticated, TLS-enforced external relay; inbound submission (port 587) requires SASL authentication, which Postfix delegates to Dovecot over a Unix socket instead of implementing its own auth backend. Dovecot stores mail as Maildir, one directory tree per domain/user, and exposes both IMAP and IMAPS.

6.4 Monitoring and centralized logging

LibreNMS (behind an `nginx/PHP-FPM` front end) provides SNMP-based device and service monitoring across both sites. Every network device and service host is configured, via the Ansible `syslog.yml` playbook, to ship its logs to a single Graylog instance, which journals and indexes them in MongoDB and exposes one searchable interface — filterable by time range, source, and severity — instead of requiring an operator to log into each device individually to investigate an incident.

6.5 Database layer

Both application databases — the Cloud platform's and the internal Employee Portal's — run PostgreSQL in a Patroni-managed primary/replica configuration rather than a bare single instance. PostgreSQL's native streaming replication handles the data path, while Patroni (coordinated through an `etcd` DCS) handles leader election and automatic failover on top of it, so losing a primary node is not a single point of failure for either application.

7. Automation

The project deliberately started from a traditionally, manually configured infrastructure and evolved toward an automated one, so the automation layer reflects a real operational workflow rather than a from-scratch Infrastructure-as-Code exercise.

7.1 Ansible — configuration management

Inventory is grouped by role:

inventory.ini

```
[Routers]
R1

[Switches]
DSW ASW1 ASW2 ASW3 ASW4 DmzSW

[NetworkDevices:children]
Routers
Switches

[AccessSwitches]
ASW1
ASW2
ASW3
ASW4
DmzSW

[Servers]
WebServer
```

with group-scoped connection variables (`network_cli/ios` for network devices, plain SSH for servers) and vaulted, AES-256-encrypted credentials per group.

7.1.1 Network playbooks

- `global_conf.yml` — console/VTY session hygiene (`exec-timeout 30`, synchronous logging) and a shared DNS resolver, pushed to every network device.
- `ntp.yml` — NTP server and timezone, with a verification step against `show ntp associations`.
- `snmp.yml` — SNMP read-only community, verified against `show snmp community`.
- `stp_feat.yml` — Rapid-PVST globally, portfast and bpduguard on access-layer edge ports.
- `syslog.yml` — points every device at the central Graylog instance and enables debug-level trap logging.

```
- name: Harden STP on access switches
hosts: AccessSwitches
tasks:
  - name: Enable Rapid-PVST globally
    cisco.ios.ios_config:
      lines:
        - spanning-tree mode rapid-pvst
  - name: Portfast + BPDU guard on the access port
    cisco.ios.ios_config:
      lines:
        - spanning-tree portfast
        - spanning-tree bpduguard enable
    parents: interface GigabitEthernet0/1
```

7.1.2 System playbooks

package_install.yml provisions the web-application hosts (nginx, Node.js 20.x, npm, pm2) so a fresh server VM is ready to receive a deployed application without any manual step.

NIC bonding for server hardware redundancy

Server-facing NIC redundancy is applied through the certified redhat.rhel_system_roles.network role, bonding two physical Ethernet ports into a single logical interface in active-backup mode:

```
- name: Configure the network
hosts: Servers
tasks:
  - name: Bond connection profile with two Ethernet ports
    ansible.builtin.include_role:
      name: redhat.rhel_system_roles.network
    vars:
      network_connections:
        # Logical bond interface
        - name: bond0
          type: bond
          interface_name: bond0
          ip:
            dhcp4: yes
            auto6: yes
          bond:
            mode: active-backup
            state: up
        # First physical port of the bond
        - name: bond0-port1
          interface_name: enp7s0
          type: ethernet
          controller: bond0
          state: up
        # Second physical port of the bond
        - name: bond0-port2
          interface_name: enp8s0
          type: ethernet
          controller: bond0
          state: up
```

In active-backup mode only one physical port carries traffic at a time; the second sits as a hot standby that takes over automatically if the first port's link fails — the same hardware-link-redundancy principle LACP EtherChannel gives the switch fabric (3.4), applied one layer down at the NIC level. This is the "hardware redundancy" leg of the project's three-axis redundancy design (Chapter 2), alongside HSRP/ECMP for network redundancy and Patroni replication for data redundancy.

7.2 Terraform — self-service compute provisioning

Terraform underpins the Cloud platform's self-service compute layer, provisioning both of its compute tiers, invoked directly by the platform's backend on every customer VM/container creation request (Chapter 8.1).

7.2.1 VM tier: VPS.tf / cloudinit.tf

VPS.tf — KVM domain over the libvirt provider

```
provider "libvirt" {
  uri = "qemu+ssh://<USER>@<HYPERVISOR_IP>/system?socket=/run/libvirt/virtqemud-sock"
}

resource "libvirt_domain" "VPS" {
  name      = var.vm_name
  memory    = var.vm_memory
  vcpu      = var.vm_cpu
  qemu_agent = true
  cpu { mode = "host-passthrough" }

  cloudinit = libvirt_cloudinit_disk.vm_init.id
  disk { volume_id = libvirt_volume.VPS_disk.id }
  network_interface { bridge = "br-fabric" }
  console { type = "pty", target_port = "0", target_type = "serial" }
}

resource "libvirt_volume" "VPS_disk" {
  name      = "${var.vm_name}.qcow2"
  pool      = var.user_id
  base_volume_name = var.os_image
  base_volume_pool = "default"
  size      = var.vm_size * 1073741824
}
```

Every tenant gets its own libvirt storage pool (`var.user_id`), so disk images are isolated per customer even though they all run on the same hypervisor. `cloudinit.tf` builds the first-boot configuration injected through that disk: user creation with an injected SSH key, baseline package installation, and enabling the QEMU guest agent so the platform can query the VM's live state and IP address. On top of this baseline, the deployed platform layers a configurable per-VM username and console password, and a second, internet-facing NIC used purely for package installation traffic — kept separate from the customer-facing interface so a VM's reachability to real customers never depends on outbound internet access being available.

7.2.2 Container tier: Container.tf

Container.tf — Docker container over the kreuzwerker/docker provider

```
provider "docker" {
  host = "ssh://<USER>@<HYPERVISOR_IP>"
}

resource "docker_image" "app" {
  name      = var.container_image
  keep_locally = true
}

resource "docker_container" "app" {
  name      = var.container_name
  image     = docker_image.app.image_id
  cpu_shares = var.cpu_shares
  memory    = var.memory
  ports { internal = var.internal_port, external = var.external_port }
  labels { label = "user_id", value = var.user_id }
}
```

Docker containers are the lighter-weight compute tier of the same platform, provisioned over the same SSH transport to the hypervisor as the VM tier, and tagged per tenant via the `user_id` label for isolation and accounting.

8. Application Layer

Two applications run on top of this infrastructure, one per site, matching the strict population separation required by the project brief: an external, customer-facing platform in the Datacenter DMZ, and an internal, staff-only portal at HQ that isn't reachable from the DMZ at all.

8.1 Cloud platform (Datacenter DMZ)

The backend (Node.js/Express) organizes its functionality into route modules under `routes/`, each corresponding to a domain of the platform:

- `auth.js` — account authentication and session issuance.
- `account.js` — account/profile and SSH-key management.
- `resources.js` — the core provisioning API: create, list, start/stop/reboot, and destroy VMs/containers; drives `services/terraform.js` on every mutation.
- `dashboard.js` / `billing.js` — account-level resource overview and usage-based billing figures.

Supporting services under `services/` do the actual infrastructure work behind those routes:

- `terraform.js` — drives `VPS.tf/Container.tf` per customer workspace; validates and content-addresses bring-your-own-image URLs (`imageValidation.js`) so the same image, requested by any customer, is downloaded once on the hypervisor and reused by every subsequent VM built from it.
- `virsh.js` — queries live domain state (running/stopped, IP address, resource stats) directly from the hypervisor, so the platform's own state is reconciled against the real hypervisor state on every request rather than trusted optimistically.
- `consoleBridge.js` — bridges a browser WebSocket to a VM's serial console over SSH, giving a real in-browser console independent of the VM's own network configuration.
- `natMap.js` — reads the private-to-public IP mapping table maintained by the Datacenter's NAT gateway (below), so the platform can show and use each VM's actual reachable address.
- `stats.js` — live CPU/memory/disk/network metrics sourced from the QEMU guest agent running inside each VM.

8.1.1 Automated public-IP exposure

Every provisioned VM's private address is automatically mapped to a public address from a dedicated pool the moment DHCP assigns it a lease, and unmapped automatically when the VM is destroyed — no manual network administration required per customer workload. This is implemented as a DHCP on-commit/on-expiry hook pair that SSHes into the Datacenter's dedicated NAT container and installs, or removes, a matching DNAT/SNAT rule pair for that specific address, recording the mapping in a flat file the platform's backend reads directly — no credentials or extra network hop needed, since the DHCP server and the backend run on the same host.

8.2 Employee Portal (HQ, internal only)

The internal portal is the single application through which Strata's staff conduct all internal work, authenticated against a centralized directory service and gated by department/role. Its backend is likewise organized into one route module per feature area:

- `auth.js / users.js` — authentication and directory-backed user management.
- `mail.js` — internal mail, integrated with the Dovecot mailbox backend (`services/dovecot.js`) rather than a separate application-level mail store.
- `documents.js` — the shared file repository, with directory-level access permissions enforced by department.
- `helpdesk.js` — IT ticketing, routed to the Support team.
- `hr.js / selfservice.js` — HR record management and the employee-facing leave/payslip/document self-service module.
- `clients.js` — the cloud-operations dashboard's view into the Datacenter platform (fabric health, provisioned workload counts).
- `announcements.js` — the company/department announcement board.
- `audit.js` — a centralized audit trail of actions taken across the portal's modules.

9. Lessons Learned and Future Work

9.1 What held up

The redundancy model proved solid end-to-end: OSPF reconverged cleanly around simulated link and device failures, the HSRP priority split kept both distribution switches carrying real traffic rather than one sitting idle, and the Terraform-driven provisioning pipeline worked exactly as intended — a customer signing up on the portal reliably ends with a running VM on the hypervisor, no manual step in between. The zone-based, default-deny firewall policy also held up cleanly under review: every permitted flow traced back to a specific, documented business need rather than a broad allow rule left over from testing.

9.2 How the project evolved

The project deliberately progressed through successive tooling phases rather than being built once end-to-end: an initial single-firewall, single-router design — still visible in the earliest addressing documentation — was replaced by the fully redundant dual-firewall/dual-router/dual-distribution-switch design once the cost of asymmetric redundancy became clear. The network build itself moved from Packet Tracer, to a full GNS3/EVE-style VM lab for maximum platform fidelity, to a Containerlab-based container implementation of the Datacenter fabric once running full router/switch VMs alongside dozens of customer VMs became impractical on a single hypervisor.

9.3 Outstanding items

Tracked for future iterations of this infrastructure:

- Per-customer network isolation at the Datacenter layer, beyond the storage-pool isolation already in place — separate segments or NAT scoping per tenant. Deferred deliberately in this iteration, since the project's goal was hands-on network/systems buildout rather than a sellable multi-tenant product; the automated public-IP mapping in 8.1.1 covers reachability but not full per-tenant network isolation.
- Layer-2 edge hardening beyond what's already deployed — Port Security (caps MAC addresses per access port, preventing CAM-table overflow/MAC-flooding attacks that force a switch into flooding all traffic like a hub), DHCP Snooping (blocks rogue DHCP servers on untrusted access ports), and Dynamic ARP Inspection (validates ARP replies against the DHCP snooping binding table, preventing ARP spoofing and the on-path MITM position it enables) — intentionally sequenced last since it depends on every access-layer VLAN/port assignment being finalized first, and out of scope for this round of testing since none of the planned attack scenarios target Layer 2.
- Full alignment with the standard cloud-computing service characteristics (on-demand self-service, broad network access, resource pooling, rapid elasticity, measured service) — most are already met by the Terraform-driven provisioning layer; measured service (usage-based accounting) is the remaining gap.
- Customer VPS disk encryption at rest (LUKS), with the encryption key held by the customer rather than the platform operator.

10. Conclusion

This report covered the design and deployment of Strata's network and systems infrastructure across two sites: a redundant enterprise campus at Headquarters and a spine-leaf compute fabric in the Cloud Datacenter, interconnected over an authenticated, encrypted site-to-site link, with a strict, explicit, default-deny zone policy and IDS sensor coverage enforced at every boundary. On top of that network, a full set of core infrastructure services — DHCP/DNS, AAA, mail, monitoring, centralized logging — was deployed and largely automated through Ansible, including hardware-level NIC bonding on servers alongside the switch-level and fabric-level redundancy mechanisms covered earlier. A Terraform-driven self-service compute layer turns the Datacenter fabric into an actual, usable IaaS offering, with real customer-facing provisioning, lifecycle management, console access, and public reachability. Together with the internal Employee Portal, this constitutes the full design-and-deployment scope of the Strata project; the attack-simulation campaign against it is documented separately as its own exercise.